



DCC

DEPARTAMENTO DE CIENCIA
DE LA COMPUTACIÓN

IIC2343

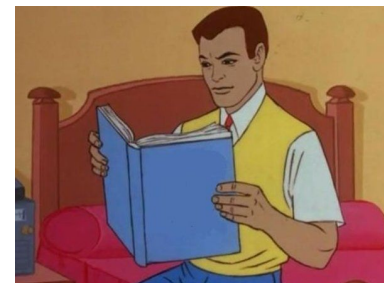
Arquitectura de Computadores

Clase 8 - Arquitectura RISC-V

Profesor: Germán Leandro Contreras Sagredo

Bibliografía

- **D. Patterson, Computer Organization and Design RISC-V Edition: The Hardware Software Interface. Morgan Kaufmann, 2020.**
 - Capítulos 2.5-2.6-2.7-2.8. Página 81, 120 en PDF.



Objetivos de la clase

- Conocer la arquitectura RISC-V a nivel general.
- Conocer el *set* de instrucciones de la ISA RISC-V.
- Conocer las convenciones de llamada utilizadas en RISC-V.
- Ver ejemplos de código en clases a través del simulador RARS.

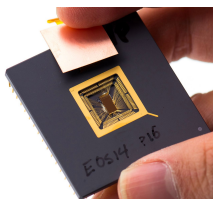
Arquitectura RISC-V (RISC “five”)

- ISA de tipo **RISC** *open source*. **No es una microarquitectura.**
- Propuesta por la Universidad de Berkeley el 2010 (pero con muchos colaboradores externos a la fecha).
- Arquitectura de tipo *load-store*. Posee dos categorías de instrucción: acceso de memoria y ALU (solo entre registros).
- Posee un diseño modular, *i.e.* que posee una ISA base y un conjunto de extensiones **opcionales**.

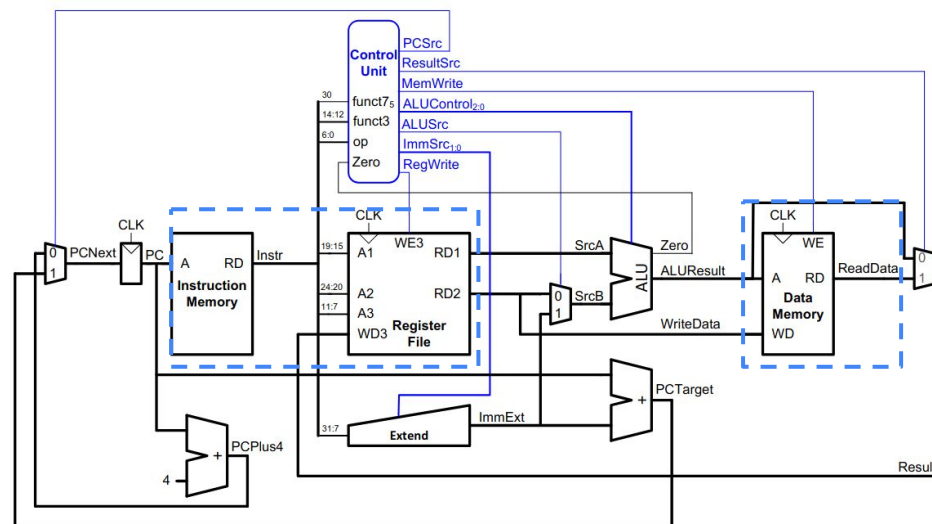


Arquitectura RISC-V - Ejemplo de diagrama reducido a un ciclo

- **Register File** posee el banco de 16 o 32 registros de la arquitectura.
- Presenta memoria de instrucciones y datos (Harvard).



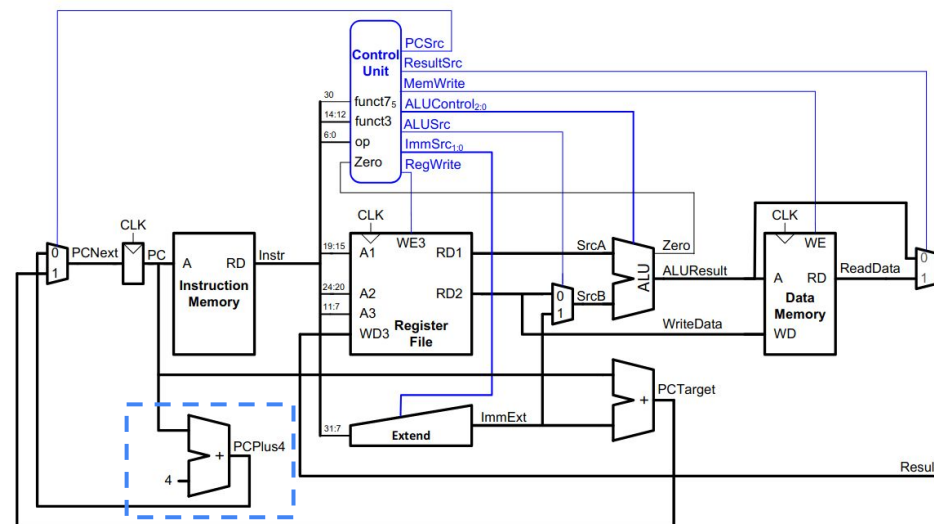
Primer prototipo (2013).



Este diagrama corresponde a una propuesta que permite implementar las instrucciones de la ISA RISC-V con ejecución en un ciclo.

Arquitectura RISC-V - Ejemplo de diagrama reducido a un ciclo

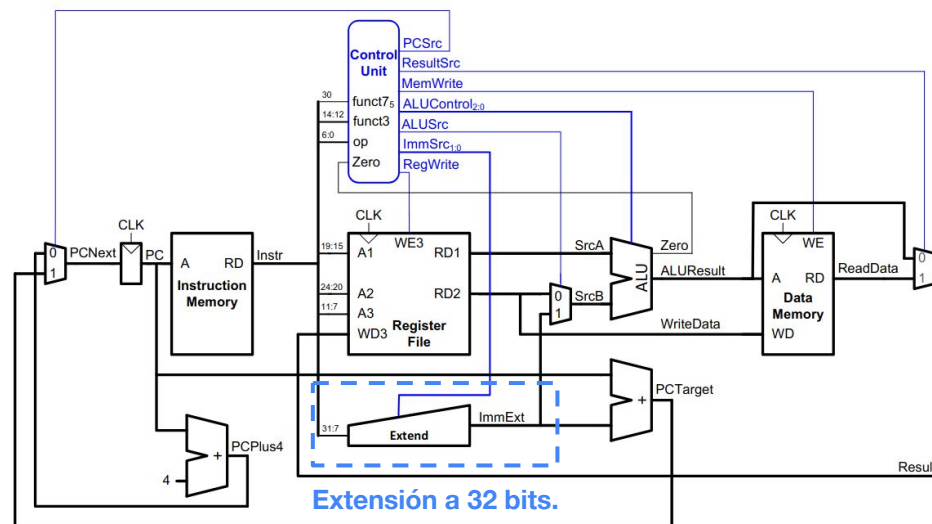
- Palabras de memoria de 8 bits (1 byte).
- Direcciones de memoria de 32 bits (4 bytes).
- Instrucciones de 32 bits (4 bytes). Ocupan 4 direcciones de memoria.



Como las instrucciones ocupan 4 direcciones de la memoria, PC + 4 apunta a la siguiente instrucción.

Arquitectura RISC-V - Ejemplo de diagrama reducido a un ciclo

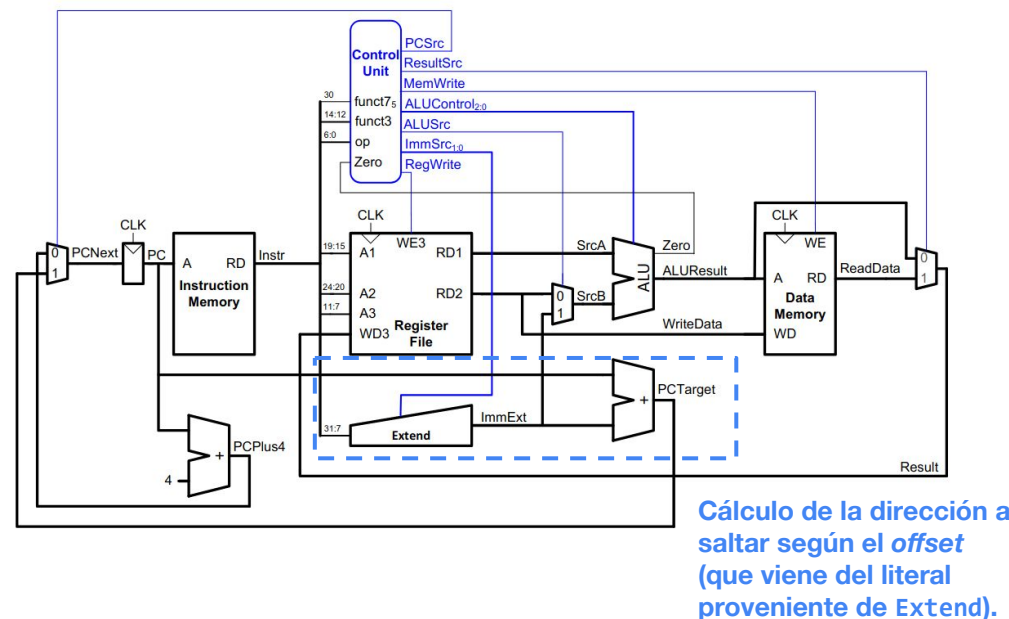
- Manejo de literales (llamados *immediates*).
- Como el literal ocupa menos de 32 bits dentro de la instrucción, **Extend** extiende su tamaño a 32 bits para operar con los valores de los registros. Esta extensión se realiza sobre el **bit de signo**.



Adicionalmente, el componente **Extend** recibe un *input* `ImmSrc` de 2 bits ya que, según el tipo de instrucción, pueden existir hasta cuatro formas de distribuir los bits del literal en ellas.

Arquitectura RISC-V - Ejemplo de diagrama reducido a un ciclo

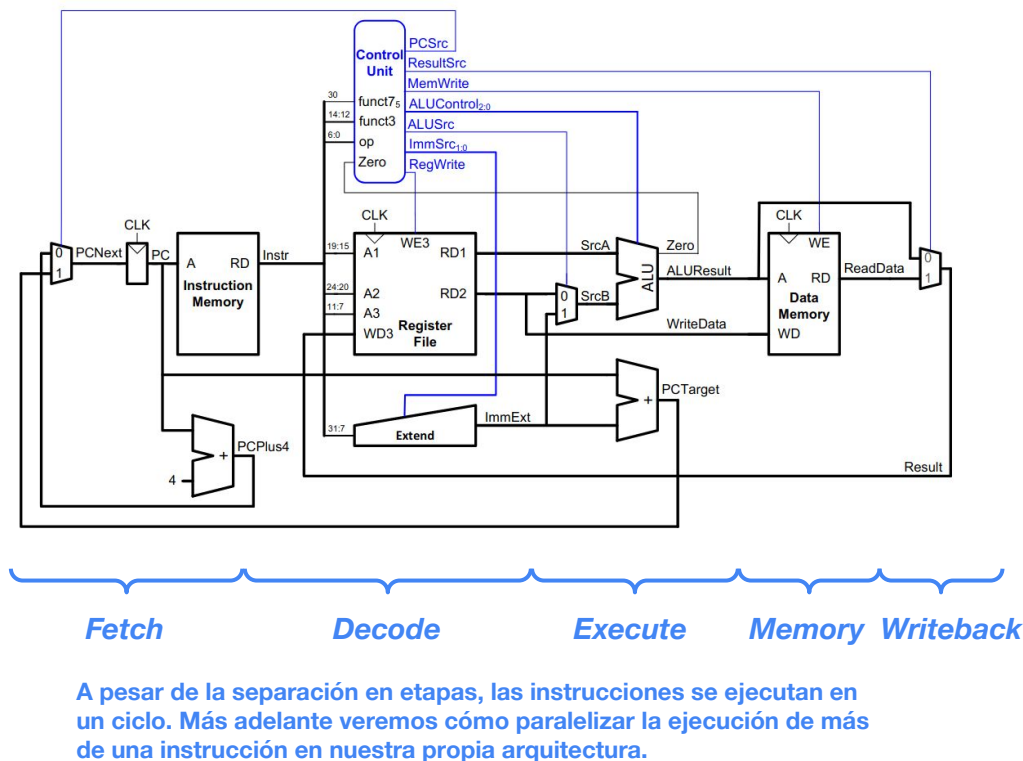
- Las instrucciones de salto, a diferencia de las vistas en el computador básico, cargan la dirección de la instrucción mediante un *offset* que se suma con el PC actual.



Arquitectura RISC-V - Ejemplo de diagrama reducido a un ciclo

Se separa en 5 etapas:

- **Fetch:** Obtención de la instrucción.
- **Decode:** Obtención de señales de control.
- **Execute:** Ejecución (ALU).
- **Memory:** Lectura o escritura en memoria.
- **Writeback:** escritura sobre el banco de registros.

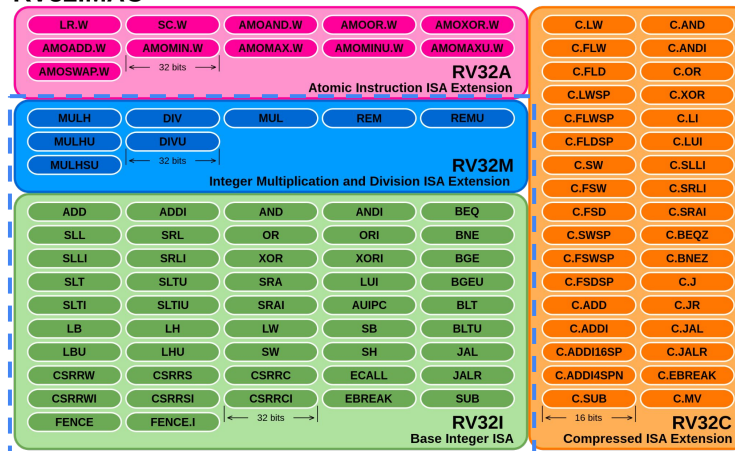


Arquitectura RISC-V

- Se considera una arquitectura **Harvard + RISC**.
- Usaremos el conjunto base de instrucciones para números enteros (RV32I) y la extensión de multiplicación y división (RV32M).



RV32IMAC



Conjunto de instrucciones base y extensiones. En nuestro caso, usaremos la ISA RV32IM: RISC-V de 32 bits con extensiones *Integer e Integer Multiplication and Division*.

Arquitectura RISC-V - Microarquitectura en detalle

- 32 registros de propósito general de 32 bits dentro de un ***Register File***.
- La arquitectura varía según los módulos usados. RV32I posee una sola unidad de ejecución (ALU).
- Direcciones de memoria: 32 bits; palabras de memoria: 8 bits. Almacenamiento ***little endian***.
- *Stack* en memoria. Registro SP apunta al **último elemento ingresado al *stack***.

Arquitectura RISC-V - ISA en detalle

La ISA de la arquitectura RISC-V es un poco más compleja que la del computador básico (a pesar de ser RISC):

- Posee instrucciones de transferencia, aritméticas, lógicas, saltos y subrutinas.
- Posee tipos de instrucción. Cada tipo posee un formato distinto (a diferencia del computador básico donde se tiene solo *opcode* y literal concatenados para toda instrucción).
- Acepta tipos de datos nativos de 8 y 32 bits (con y sin signo).
- Posee un solo tipo de direccionamiento: **indirecto + *offset***.

Arquitectura RISC-V - ISA en detalle

Listado de registros

Registro(s)	Mnemotecnia ABI	Descripción
x0	zero	Registro cero. Almacena este valor y no cambia . Ignora las escrituras.
x1	ra	<i>Return Address</i> . Almacena la dirección de retorno de las subrutinas.
x2	sp	<i>Stack Pointer</i> , apunta al último elemento almacenado.
x3	gp	<i>Global Pointer</i> , apunta al segmento de memoria donde se almacenan las variables globales.
x4	tp	<i>Thread Pointer</i> , apunta al segmento de memoria donde se almacenan las variables de un <i>thread</i> para aplicaciones de múltiples <i>threads</i> .
x5-x7, x28-x31	t0-t6	Registros temporales. Pierden su valor entre llamados de subrutinas.
x8-x9, x18-x27	s0-s11	Registros guardados (<i>saved</i>). Preservan su valor entre llamados de subrutinas.
x10-x17	a0-a7	Registros para argumentos de subrutinas.
x10-x11	a0-a1	Si bien son de argumentos de subrutinas, también se utilizan para almacenar valores de retorno.

A nivel de Assembly, usaremos los nombres mnemotécnicos para referirnos a estos registros, respetando la ABI (Application Binary Interface).

Si bien todos son de propósito general, por convención los usaremos según las descripciones de este listado (más información en las siguientes diapositivas).

Arquitectura RISC-V - ISA en detalle

Listado de directivas de Assembler

Directiva de Assembler	Descripción
<code>.text</code>	Segmento de texto (código).
<code>.data</code>	Sección de datos global.
<code>.bss</code>	Sección de datos globales inicializados en 0.
<code>.section .foo</code>	Sección llamada "foo".
<code>.align N</code>	Alinear el siguiente dato/instrucción a 2^N bytes de memoria.
<code>.balign N</code>	Alinear el siguiente dato/instrucción a N bytes de memoria.
<code>.globl sym</code>	<i>Label</i> sym se vuelve global.
<code>.string "str"</code>	Almacena el <i>string</i> "str" en memoria.
<code>.word w1, w2, ..., wN</code>	Almacena N valores de 32 bits en palabras de memoria sucesivas.
<code>.byte w1, w2, ..., wN</code>	Almacena N valores de 8 bits en bytes de memoria sucesivos.
<code>.space N</code>	Reserva N bytes para almacenar una variable.
<code>.equ name, constant</code>	Define el símbolo name con valor constant.
<code>.end</code>	Término del código Assembly.

Las directivas de un Assembler consisten en órdenes para que este tome ciertas acciones o realice cambios de configuración.

¡No son instrucciones ni se traducen a código de máquina!

* `.align`, `.balign` agregan *padding* de bytes "basura" en memoria de forma que la inserción de la siguiente variable quede almacenada en una dirección "alineada" (generalmente una dirección par para optimizar direccionamiento).

Arquitectura RISC-V - ISA en detalle

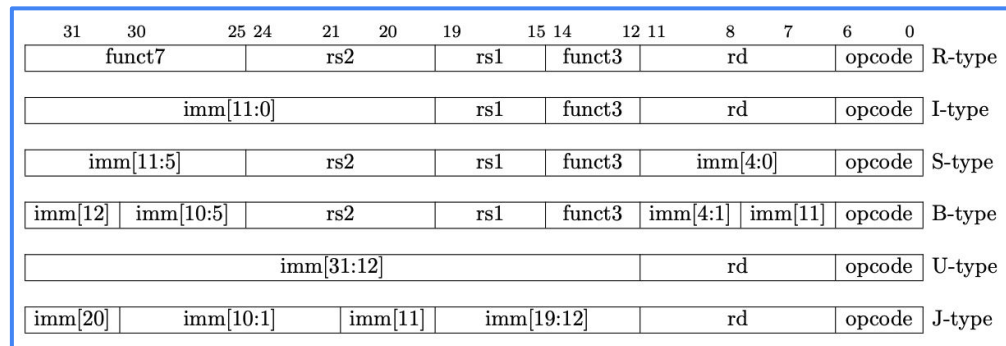
Tipos de instrucción

- I-Type: *Immediate Type*. Utilizan al menos un literal de operando.
- R-Type: *Register Type*. Utilizan solo registros como operandos.
- S-Type: *Store Type*. Almacenamiento en la memoria de datos.
- B-Type: *Branching Type*, variante de S-Type. Saltos condicionales a través de *offsets*.
- U-Type: *Upper Immediate Type*. Permite cargar en registros literales de más de 12 bits (límite de instrucciones I-Type).
- J-Type: *Jump Type*, variante de U-Type. Saltos incondicionales para las subrutinas.

Arquitectura RISC-V - ISA en detalle

Formato de tipos de instrucción

- Instrucciones de 32 bits.
- `imm` = literal (*immediate*).
- `rs1`, `rs2` = registros de operandos.
- `rd` = registro de destino.
- `opcode` = identificador de la instrucción.
- `funct7`, `funct3` = Bits adicionales que, en conjunto con el `opcode`, definen la operación completa a ejecutar.



Arquitectura RISC-V - ISA en detalle

Resumen de instrucciones - Operaciones aritméticas

Mnemotecnia	Instrucción	Tipo	Descripción
ADD rd, rs1, rs2	Adición	R	$rd \leftarrow rs1 + rs2$
SUB rd, rs1, rs2	Sustracción	R	$rd \leftarrow rs1 - rs2$
ADDI rd, rs1, imm12	Adición de literal	I	$rd \leftarrow rs1 + imm12$
SLT rd, rs1, rs2	Configurar “menor a”	R	$rd \leftarrow rs1 < rs2 ? 1 : 0$
SLTI rd, rs1, rs2	Configurar “menor a” literal	I	$rd \leftarrow rs1 < imm12 ? 1 : 0$
SLTU rd, rs1, rs2	Configurar “menor a” sin signo	R	$rd \leftarrow rs1 < rs2 ? 1 : 0$
SLTIU rd, rs1, imm12	Configurar “menor a” literal sin signo	I	$rd \leftarrow rs1 < imm12 ? 1 : 0$
LUI rd, imm20	Cargar literal “superior” (20 bits)	U	$rd \leftarrow imm20 \ll 12$ (SHL 12)
AUIPC rd, imm20	Sumar literal “superior” a PC (20 bits)	U	$rd \leftarrow PC + imm20 \ll 12$ (SHL 12)

* Para restar un literal, usamos ADDI con un literal negativo.

Arquitectura RISC-V - ISA en detalle

Resumen de instrucciones - Operaciones lógicas 1/2

Mnemotecnia	Instrucción	Tipo	Descripción
AND rd, rs1, rs2	Operación AND	R	$rd \leftarrow rs1 \& rs2$
OR rd, rs1, rs2	Operación OR	R	$rd \leftarrow rs1 \mid rs2$
XOR rd, rs1, rs2	Operación XOR	R	$rd \leftarrow rs1 \wedge rs2$
ANDI rd, rs1, imm12	Operación AND con literal	I	$rd \leftarrow rs1 \& imm12$
ORI rd, rs1, imm12	Operación OR con literal	I	$rd \leftarrow rs1 \mid imm12$
XORI rd, rs1, imm12	Operación XOR con literal	I	$rd \leftarrow rs1 \wedge imm12$

Arquitectura RISC-V - ISA en detalle

Resumen de instrucciones - Operaciones lógicas 2/2

Mnemotecnia	Instrucción	Tipo	Descripción
SLL rd, rs1, rs2	Operación <i>shift left</i> lógico	R	$rd \leftarrow rs1 \ll rs2$
SRL rd, rs1, rs2	Operación <i>shift right</i> lógico	R	$rd \leftarrow rs1 \gg rs2$
SRA rd, rs1, rs2	Operación <i>shift right</i> aritmético	R	$rd \leftarrow rs1 \ggg rs2$
SLLI rd, rs1, shamt	Operación <i>shift left</i> lógico con literal	I	$rd \leftarrow rs1 \ll shamt$
SRLI rd, rs1, shamt	Operación <i>shift right</i> lógico con literal	I	$rd \leftarrow rs1 \gg shamt$
SRAI rd, rs1, shamt	Operación <i>shift right</i> aritmético con literal	I	$rd \leftarrow rs1 \ggg shamt$

* *shamt* o *shift amount* es la cantidad de *shifts* a realizar y se codifica como un entero a partir de los 5 bits menos significativos del literal (`imm12[4:0]`).

Arquitectura RISC-V - ISA en detalle

Resumen de instrucciones - Operaciones de carga y almacenamiento

Mnemotecnia	Instrucción	Tipo	Descripción
LW rd, imm12(rs1)	Cargar word (32 bits)	I	$rd \leftarrow \text{mem}[rs1 + \text{imm12}]$
LH rd, imm12(rs1)	Cargar <i>half word</i> (16 bits)	I	$rd \leftarrow \text{mem}[rs1 + \text{imm12}]$
LB rd, imm12(rs1)	Cargar byte (8 bits)	I	$rd \leftarrow \text{mem}[rs1 + \text{imm12}]$
LWU rd, imm12(rs1)	Cargar <i>word</i> sin signo (32 bits)	I	$rd \leftarrow \text{mem}[rs1 + \text{imm12}]$
LHU rd, imm12(rs1)	Cargar <i>half word</i> sin signo (16 bits)	I	$rd \leftarrow \text{mem}[rs1 + \text{imm12}]$
LBU rd, imm12(rs1)	Cargar byte sin signo (8 bits)	I	$rd \leftarrow \text{mem}[rs1 + \text{imm12}]$
SW rs2, imm12(rs1)	Almacenar word (32 bits)	S	$rs2 \rightarrow \text{mem}[rs1 + \text{imm12}]$
SH rs2, imm12(rs1)	Almacenar <i>half word</i> (16 bits)	S	$rs2(15:0) \rightarrow \text{mem}[rs1 + \text{imm12}]$
SB rs2, imm12(rs1)	Almacenar byte (8 bits)	S	$rs2(7:0) \rightarrow \text{mem}[rs1 + \text{imm12}]$

* Para direccionar en LW y SW, se usa el formato offset(x), donde la dirección se almacena en el registro x. Ejemplo: 4(sp)

Arquitectura RISC-V - ISA en detalle

Resumen de instrucciones - Saltos y subrutinas

Mnemotecnia	Instrucción	Tipo	Descripción
BEQ rs1, rs2, imm12	Salto con condición “igual”	B	if rs1 == rs2: PC $\leftarrow$ PC + imm12
BNE rs1, rs2, imm12	Salto con condición “distinto”	B	if rs1 != rs2: PC $\leftarrow$ PC + imm12
BGE rs1, rs2, imm12	Salto con condición “mayor o igual”	B	if rs1 >= rs2: PC $\leftarrow$ PC + imm12
BGEU rs1, rs2, imm12	Salto con condición “mayor o igual” sin signo	B	if rs1 >= rs2: PC $\leftarrow$ PC + imm12
BLT rs1, rs2, imm12	Salto con condición “menor”	B	if rs1 < rs2: PC $\leftarrow$ PC + imm12
BLTU rs1, rs2, imm12	Salto con condición “menor” sin signo	B	if rs1 < rs2: PC $\leftarrow$ PC + imm12
JAL rd, imm20	Salto incondicional con “enlace”	J	rd $\leftarrow$ PC+4; PC $\leftarrow$ PC + imm20
JALR rd, imm12(rs1)	Salto incondicional con “enlace” a registro	I	rd $\leftarrow$ PC+4; PC $\leftarrow$ rs1 + imm12

* En estos casos, el literal representa el *offset* para llegar a la instrucción deseada desde el *Program Counter*. A nivel de código, se observa como el *label* de la dirección a saltar.

Arquitectura RISC-V - ISA en detalle

Resumen de instrucciones - Pseudo-instrucciones 1/2

Mnemotecnia	Instrucción	Instrucción(es) base
LI rd, imm12	Cargar literal en registro que utiliza ≤ 12 bits	ADDI rd, zero, imm12
LI rd, imm	Cargar literal en registro que utiliza > 12 bits	LUI rd, imm[31:12]; ADDI rd, rd, imm[11:0]
LA rd, sym	Cargar dirección en registro	AUIPC rd, sym[31:12]; ADDI rd, rd, sym[11:0]
MV rd, rs	Copiar registro	ADDI rd, rs, 0
NOT rd, rs	Complemento de 1	XORI rd, rs, -1
NEG rd, rs	Complemento de 2	SUB rd, zero, rs
BGT rs1, rs2, offset	Salto si $rs1 > rs2$	BLT rs2, rs1, offset
BLE rs1, rs2, offset	Salto si $rs1 \leq rs2$	BGE rs2, rs1, offset
BGTU rs1, rs2, offset	Salto si $rs1 > rs2$ sin signo	BLTU rs2, rs1, offset
BLEU rs1, rs2, offset	Salto si $rs1 \leq rs2$ sin signo	BGEU rs2, rs1, offset

* Las pseudo-instrucciones mnemotécnicas se traducen a las instrucciones reales de RISC-V. Esto ayuda a tener instrucciones de operaciones útiles en un lenguaje más sencillo de entender.

Arquitectura RISC-V - ISA en detalle

Resumen de instrucciones - Pseudo-instrucciones 2/2

Mnemotecnia	Instrucción	Instrucción(es) base
BEQZ rs1, offset	Salto si $rs1 = 0$	BEQ rs1, zero, offset
BNEZ rs1, offset	Salto si $rs1 \neq 0$	BNE rs1, zero, offset
BGEZ rs1, offset	Salto si $rs1 \geq 0$	BGE rs1, zero, offset
BLEZ rs1, offset	Salto si $rs1 \leq 0$	BGE zero, rs1, offset
BGTZ rs1, offset	Salto si $rs1 > 0$	BLT zero, rs1, offset
J offset	Salto incondicional	JAL zero, offset
CALL offset12	Llamado a subrutina (dirección ≤ 12 bits)	JALR ra, ra, offset12
CALL offset*	Llamado a subrutina (dirección > 12 bits)	AUIPC ra, offset[31:12]; JALR ra, ra, offset[11:0]
RET	Retorno de la subrutina	JALR zero, 0(ra)
NOP	No se realiza ninguna operación	ADDI zero, zero, 0

* Esta es la pseudo-instrucción que ocupa el emulador a utilizar, RARS.

Arquitectura RISC-V - ISA en detalle

Ejemplo de código - Multiplicación

```
a = 10
b = 200
res = 0
while (a > 0):
    res += b
    a -= 1
print(res)
```

Programa en pseudocódigo
(Python).

```
.text
main:
    addi t0, zero, 10          # t0 = 10
    addi t1, zero, 200         # t1 = 200
    addi t2, zero, 0           # t2 = 0
mul_loop:
    beq t0, zero, end          # if t0 = 0 jmp end
    add t2, t2, t1             # t2 += t1
    addi t0, t0, -1            # t0 += -1
    jal zero, mul_loop         # jmp mul_loop
end:
```

Programa en RISC-V.

Arquitectura RISC-V - ISA en detalle

Ejemplo de código - Multiplicación

```
a = 10
b = 200
res = 0
while (a > 0):
    res += b
    a -= 1
print(res)
```

Programa en pseudocódigo
(Python).

```
.text
main:
    li t0, 10           # t0 = 10
    li t1, 200          # t1 = 200
    li t2, 0            # t2 = 0
mul_loop:
    beq t0, zero, end   # if t0 = 0 jmp end
    add t2, t2, t1       # t2 += t1
    addi t0, t0, -1      # t0 += -1
    j mul_loop          # jal zero, mul_loop
end:
```

Programa en RISC-V equivalente al anterior con uso de
pseudo-instrucciones.

Arquitectura RISC-V - ISA en detalle

Resumen de instrucciones - Extensión M

Mnemotecnia	Instrucción	Tipo	Descripción
MUL rd, rs1, rs2	32 bits menos significativos del producto.	R	$rd \leftarrow (rs1 * rs2)[31:0]$
MULH rd, rs1, rs2	32 bits más significativos del producto (rs1, rs2 con signo).	R	$rd \leftarrow (rs1 * rs2)[63:32]$
MULHSU rd, rs1, rs2	32 bits más significativos del producto (rs1 con signo, rs2 sin signo).	R	$rd \leftarrow (rs1 * rs2)[63:32]$
MULHU rd, rs1, rs2	32 bits más significativos del producto (rs1, rs2 sin signo).	R	$rd \leftarrow (rs1 * rs2)[63:32]$
DIV rd, rs1, rs2	División con signo	R	$rd \leftarrow rs1 / rs2$
DIVU rd, rs1, rs2	División sin signo	R	$rd \leftarrow rs1 / rs2$
REM rd, rs1, rs2	Resto con signo	R	$rd \leftarrow rs1 \% rs2$
REMU rd, rs1, rs2	Resto sin signo	R	$rd \leftarrow rs1 \% rs2$

Arquitectura RISC-V - ISA en detalle

Ejemplo de código - Multiplicación con extensión M

```
.text
main:
    addi t0, zero, 10      # t0 = 10
    addi t1, zero, 200     # t1 = 200
    mul  t2, t0, t1        # t2 = t0 * t1 = 2000
```

Si agregamos la extensión RV31M al conjunto de instrucciones base RV31I, entonces el código se simplifica de manera significativa (similar a lo que ocurre con las operaciones MUL y DIV de la arquitectura x86).



Arquitectura RISC-V - ISA en detalle

Instrucción ECALL (*Environment Call*)

- Instrucción especial de tipo I que permite realizar una solicitud al entorno de ejecución (sistema operativo).
- Su uso varía según la implementación, pero en este curso nos basaremos en la implementación del simulador de RISC-V [RARS](#) (*RISC-V Assembler And Runtime System*).

Arquitectura RISC-V - ISA en detalle

Instrucción ECALL (*Environment Call*)

- Se hace uso de un código que indica la llamada a efectuar. Este se almacena en el registro a7 y los argumentos en a0-a6.
- Existen [múltiples funciones](#), pero usaremos dos:
 - `PrintInt(code=1)`: Impresión de valor entero en consola. Solo recibe un argumento en a0, el número a imprimir.
 - `Exit(code=10)`: Término de programa. No recibe argumentos y asume que el código de término es 0 (sin errores).

Arquitectura RISC-V - ISA en detalle

Ejemplo de código - Impresión de número y salida del programa

```
.text
main:
    addi a0, zero, 11      # a0 = 11
    addi a7, zero, 1       # a7 = 1 (PrintInt)
    ecall                  # Imprime 11 en consola
    addi a7, zero, 10      # a7 = 10 (Exit)
    ecall                  # Termina el programa
```

Ejemplo de programa que imprime en consola el número 11. Se incluye además el resultado del simulador RARS. Cabe destacar que RARS no reconoce la directiva `.end`, por lo que es necesario usar `ecall` para el término del programa.



Arquitectura RISC-V - ISA en detalle

Ejemplo de código - Multiplicación con extensión M en RARS

```
.text
main:
    addi t0, zero, 10      # t0 = 10
    addi t1, zero, 200     # t1 = 200
    mul  t2, t0, t1        # t2 = t0 * t1 = 2000
    add  a0, zero, t2      # a0 = t2
    addi a7, zero, 1       # a7 = 1 (PrintInt)
    ecall                  # Print a0 = 2000
    addi a7, zero, 10      # a7 = 10 (Exit)
    ecall                  # Exit
```



Ejemplo de código de multiplicación ejecutable en RARS.

Arquitectura RISC-V - Convención de llamada

Se hace uso de convenciones de llamada para estandarizar la forma en la que se ejecutan las subrutinas en un programa.

En el caso de RISC-V, se hace uso de una convención propia donde el almacenamiento de parámetros, valores de retorno y dirección de retorno **recae en los registros** y no en el *stack*. Este último se utiliza como respaldo para **preservar el valor de los registros entre llamadas**.

Arquitectura RISC-V - Convención de llamada

- El registro *ra* (x1) almacena la dirección de retorno.
- Los argumentos de una subrutina se almacenan en los registros *a0-a7* (x10-x17).
- Si existen valores de retorno, se almacenan en los registros *a0-a1* (x10-x11).
- Se hace uso del *stack* con el registro *sp* (x2). Se puede utilizar para incluir argumentos adicionales, variables locales y **respaldo de registros**.

Arquitectura RISC-V - Convención de llamada

El encargado de respaldar el valor de los registros con el llamado de una subrutina varía según el tipo de registro. En este caso, el encargado puede ser quien llama a la subrutina (*caller*) o la subrutina llamada (*callee*).

El respaldo de los registros se realiza a través del *stack pointer*, modificando su valor para reservar valores en la memoria de *stack*.

Registro(s)	Mnemotecnia ABI	Encargado del respaldo
x0	zero	-
x1	ra	<i>Caller</i>
x2	sp	<i>Callee</i>
x3	gp	-
x4	tp	-
x5-x7, x28-x31	t0-t6	<i>Caller</i>
x8-x9, x18-x27	s0-s11	<i>Callee</i>
x10-x17	a0-a7	<i>Caller</i>
x10-x11	a0-a1	<i>Caller</i>

Arquitectura RISC-V - Convención de llamada

Ejemplo de código - Duplicación de elementos de un arreglo

```

.data
len: .word 5
arr: .word 198, 137, 42, 63, 175
.text
start:
    addi s0, zero, 4           # s0 = 4 bytes por dirección
    lw s1, len                 # s1 = largo del arreglo
    addi t0, zero, 0           # Contador (i)
    la t1, arr                  # t1 = dirección del arreglo (inicialmente arr[0])
    mul t2, s0, s1              # t2 = s0 * s1 = bytes que ocupa el arreglo de entrada
    add t2, t2, t1              # t2 += t1 = primera dirección que podemos usar de .data
    while:
        lw a0, 0(t1)           # a0 = arr[i]
        addi sp, sp, -8         # Respalamos t0 y ra (caller-saved).
        sw t0, 0(sp)
        sw ra, 4(sp)
        jal ra, double_give_next_person
        lw t0, 0(sp)           # Recuperamos t0 y ra y restauramos el stack
        lw ra, 4(sp)
        addi sp, sp, 8
        sw a0, 0(t2)           # out[i] = a0
        addi t0, t0, 1         # t0 += 1
        beq t0, s1, end        # Termina cuando se recorre todo el arreglo
        add t1, t1, s0          # t1 += s0 = dirección arr[i+1]
        add t2, t2, s0          # t2 += s0 = dirección out[i+1]
        jal zero, while
    double_give_next_person:
        add t0, zero, a0
        add a0, a0, t0          # a0 = a0 + t0 = 2 * a0
        jalr zero, 0(ra)
    end:
        addi a7, zero, 10
        ecall

```

Programa en RISC-V que duplica los elementos de un arreglo y los almacena en las direcciones posteriores a dicha variable. El respaldo de registros *caller-saved* en la convención de llamada se realiza de la siguiente forma:

1. Los registros se respaldan en el *stack* antes de la llamada. Para ello, se desplaza el registro *sp* según la cantidad de bytes a respaldar (4 bytes por registro) y luego se almacenan a través de direccionamiento indirecto por registro *sp* con *offset*. Esto se conoce como el **preámbulo**.
2. Finalizada la llamada, se restauran los registros desde el *stack* y se restablece el valor de *sp*. Esto se conoce como el **prólogo**.

Data Segment							
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)
0x10010000	5	198	137	42	63	175	396
0x10010020	84	126	350	0	0	0	0

Resultado de la ejecución en el segmento de datos en RARS.

Arquitectura RISC-V - Convención de llamada

Ejemplo de código - Duplicación de elementos de un arreglo

```
.data
len: .word 5
arr: .word 198, 137, 42, 63, 175
.text
start:
    addi s0, zero, 4           # s0 = 4 bytes por dirección
    lw s1, len                 # s1 = largo del arreglo
    addi t0, zero, 0           # Contador (i)
    la t1, arr                 # t1 = dirección del arreglo (inicialmente arr[0])
    mul t2, s0, s1             # t2 = s0 * s1 = bytes que ocupa el arreglo de entrada
    add t2, t2, t1             # t2 += t1 = primera dirección que podemos usar de .data
    while:
        lw a0, 0(t1)           # a0 = arr[i]
        addi sp, sp, -8         # Respaldamos t0 y ra (caller-saved).
        sw t0, 0(sp)
        sw ra, 4(sp)
        jal ra, double_give_next_person
        lw t0, 0(sp)           # Recuperamos t0 y ra y restauramos el stack
        lw ra, 4(sp)
        addi sp, sp, 8
        sw a0, 0(t2)           # out[i] = a0
        addi t0, t0, 1         # t0 += 1
        beq t0, s1, end        # Termina cuando se recorre todo el arreglo
        add t1, t1, s0         # t1 += s0 = dirección arr[i+1]
        add t2, t2, s0         # t2 += s0 = dirección out[i+1]
        jal zero, while
    double_give_next_person:
        add t0, zero, a0
        add a0, a0, t0         # a0 = a0 + t0 = 2 * a0
        jalr zero, 0(ra)
    end:
        addi a7, zero, 10
        ecall
```

En este mismo ejemplo, es importante destacar que el registro ra podría no respaldarse y que se asegure el funcionamiento correcto del programa. No obstante, por convención es buena práctica hacerlo de todas formas ya que el programa podría ser ejecutado mediante una llamada desde otro archivo. Si no se respaldara ra, **se perdería la dirección de retorno para el programa que ejecuta este ejemplo**. El no respaldo de ra generará problemas **siempre que haya llamados de subrutinas anidados**.

Arquitectura RISC-V - Convención de llamada

Ejemplo de código - Duplicación de elementos de un arreglo

```

.data
len: .word 5
arr: .word 198, 137, 42, 63, 175
.text
start:
    addi t0, zero, 4           # s0 = 4 bytes por dirección
    lw t2, len                 # s1 = largo del arreglo
    addi s0, zero, 0           # Contador (i)
    la s1, arr                 # t1 = dirección del arreglo (inicialmente arr[0])
    mul s2, t0, t2              # t2 = s0 * s1 = bytes que ocupa el arreglo de entrada
    add s2, s2, s1              # t2 += t1 = primera dirección que podemos usar de .data
    while:
        lw a0, 0(s1)           # a0 = arr[i]
        # Preámbulo {
        addi sp, sp, -4         # Respalamos ra (callee-saved)
        sw ra, 0(sp)           # Recuperamos ra y restauramos el stack
        jal ra, double_give_next_person
        lw ra, 0(sp)
        # Prólogo {
        addi sp, sp, 4         # out[i] = a0
        sw a0, 0(s2)           # s0 += 1
        addi s0, s0, 1         # Termina cuando se recorre todo el arreglo
        beq s0, t2, end        # s1 += t0 = dirección arr[i+1]
        add s1, s1, t0         # s2 += t0 = dirección out[i+1]
        add s2, s2, t0
        jal zero, while
    double_give_next_person:
        # Preámbulo {
        addi sp, sp, -4         # Respalamos s0 (callee-saved)
        sw s0, 0(sp)
        add s0, zero, a0
        add a0, a0, s0          # a0 = a0 + s0 = 2 * a0
        # Prólogo {
        lw s0, 0(sp)           # Recuperamos s0 y restauramos el stack
        addi sp, sp, 4
        jalr zero, 0(ra)
    end:
    addi a7, zero, 10
    ecall

```

Veamos el mismo ejemplo pero invirtiendo los registros *s** por registros *t**. El respaldo de registros *callee-saved* en la convención de llamada se realiza de la siguiente forma:

1. Los registros se respaldan en el *stack* al comienzo de la llamada al igual que con los registros *caller-saved*. Esto también es un **preámbulo**.
2. Antes de finalizar la llamada con *ret*, se restauran los registros desde el *stack* y se restablece el valor de *sp* al igual que con los registros *caller-saved*. Esto también es un **prólogo**.

En este caso seguimos respaldando el registro *ra* por ser *callee-saved* y por lo comentado en la diapositiva anterior.

Data Segment									
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)	
0x10010000	5	198	137	42	63	175	396	274	
0x10010020	84	126	350	0	0	0	0	0	

Resultado de la ejecución en el segmento de datos en RARS.

Arquitectura RISC-V - Convención de llamada

Ejemplo de código - Duplicación de elementos de un arreglo

```
.data
len: .word 5
arr: .word 198, 137, 42, 63, 175
.text
start:
    li s0, 4                # s0 = 4 bytes por dirección
    lw s1, len              # s1 = largo del arreglo
    li t0, 0                # Contador (i)
    la t1, arr              # t1 = dirección del arreglo (inicialmente arr[0])
    mul t2, s0, s1          # t2 = s0 * s1 = bytes que ocupa el arreglo de entrada
    add t2, t2, t1          # t2 += t1 = primera dirección que podemos usar de .data
    while:
        lw a0, 0(t1)        # a0 = arr[i]
        addi sp, sp, -12
        sw t0, 0(sp)        # Respaldamos t0, t1 y ra (caller-saved). t1 se respalda
        sw t1, 4(sp)        # aunque no se use porque call lo modifica con la
        sw ra, 8(sp)        # dirección de retorno en RARS
        call double_give_next_person
        lw t0, 0(sp)        # Recuperamos t0, t1 y ra y restauramos el stack
        lw t1, 4(sp)
        lw ra, 8(sp)
        addi sp, sp, 12
        sw a0, 0(t2)        # out[i] = a0
        addi t0, t0, 1      # t0 += 1
        beq t0, s1, end     # Termina cuando se recorre todo el arreglo
        add t1, t1, s0       # t1 += s0 = dirección arr[i+1]
        add t2, t2, s0       # t2 += s0 = dirección out[i+1]
        j while
    double_give_next_person:
        mv t0, a0
        add a0, a0, t0      # a0 = a0 + t0 = 2 * a0
        ret
end:
    li a7, 10
    ecall
```

IMPORTANTE: En este ejemplo, se pudo haber hecho uso de pseudo-instrucciones como se observa a continuación. No obstante, RARS tiene la particularidad de **usar el registro `t1` como respaldo de la dirección de retorno al ejecutar la instrucción `call`**, además de realizar el respaldo en `ra`. Por lo tanto, si se llegara a usar esta instrucción, también tendríamos que respaldar el registro `t1` como *caller-saved* para asegurar que su valor sea el esperado después del retorno. Si no se respaldara, esto resultaría en el **cálculo incorrecto de la dirección de memoria del segundo elemento del arreglo en este ejemplo**.

Arquitectura RISC-V - Convención de llamada

Ejemplo de código - Factorial, función recursiva

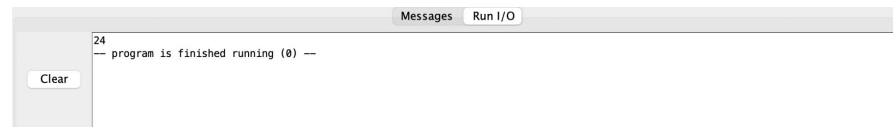
```
.data
N: .word 4           # N = Argumento de factorial. Calcularémos N! = 4
.text
main:
    addi sp, sp, -4   # Reservamos 4 bytes en el stack
    sw   ra, 0(sp)    # Respalamos ra
    la   t0, N        # Dirección de memoria de N
    lw   t0, 0(t0)    # Valor de N
    add  a0, zero, t0  # Argumento 0 = valor de N
    jal  ra, factorial # factorial(N)
    addi a7, zero, 1   # Llamada de sistema: print int
    ecall             # Valor en consola: 24 (a0, valor de retorno)
    lw   ra, 0(sp)    # Restauramos ra
    addi sp, sp, 4     # Restauramos el stack
    addi a7, zero, 10  # Llamada de sistema: exit
    ecall

factorial:
    addi sp, sp, -8   # Reservamos 8 bytes en el stack
    sw   ra, 0(sp)    # Respalamos ra
    sw   a0, 4(sp)    # Respalamos N
    bge  zero, a0, fact_zero # if (N > 0){
    addi a0, a0, -1    #   N -= 1
    jal  ra, factorial #   (N-1)! = factorial(N-1)
    lw   t0, 4(sp)    #   Recuperamos N
    mul  a0, a0, t0    #   N! = N * (N-1)! = N * factorial(N-1)
    jal  zero, fact_end # }
    fact_zero:        # else {
    addi a0, zero, 1   #   N! = 1
    fact_end:         # }
    lw   ra, 0(sp)    # Restauramos solo ra, a0 ahora posee el retorno
    addi sp, sp, 8     # Restauramos el stack
    jalr zero, 0(ra)
```

Programa en RISC-V que calcula el valor del factorial de la variable N .

Ahora, el *callee* también actúa como *caller* por la **recursión** y respalda tanto el registro ra como el registro de argumento $a0$. Además, el callee se encarga de restaurar el registro sp en cada llamada, **lo que asegura la consistencia de su valor al realizar el último retorno**. En este caso no se restaura $a0$ ya que este posee el valor de retorno a ser utilizado en la impresión de la consola.

Si se hubieran utilizado registros s^* , también habrían sido respaldados por el *callee* como en el ejemplo anterior.



Arquitectura RISC-V - Resumen de la convención de llamada

- Si usamos los registros t^* tanto fuera como dentro de una subrutina, los respaldamos en el *stack* antes del llamado y los restauramos posterior a este.
- Si usamos los registros s^* tanto fuera como dentro de una subrutina, los respaldamos en el *stack* al principio de la subrutina y los recuperamos justo antes del retorno.
- Si usamos los registros a^* , los respaldamos fuera de la subrutina pero los recuperamos siempre que no interfieran con el valor de retorno ($a2 - a7$).
- Por seguridad, siempre respaldamos el registro ra antes de un llamado.

Arquitectura RISC-V - *Cheat sheets*

A continuación, se dejan dos *cheat sheets* útiles que resumen todas las instrucciones que pueden utilizar en RISC-V:

- [Cheat Sheet 1](#)
- [Cheat Sheet 2](#)



Actividad en clase - Instalación de RARS

Antes de hacer ejercicios, realizaremos la instalación de RARS para poder ejecutar código RISC-V en nuestros computadores.

Para ello, es necesario instalar Java 8 o superior y seguir las instrucciones del [repositorio principal](#). Asegúrese de instalar: (1) el último lanzamiento estable (detallado en el README); y (2) de **instalar el archivo .jar adjunto, no clonar el repositorio.**



Actividad en clase - Instalación de Java 8

- **Windows:** Descargar Java 8 en [el siguiente enlace](#) y ejecutar el archivo .jar.
- **Mac**
 - **Chip M1:** Seguir [el siguiente tutorial](#).
 - **Modelos previos:** Descargar Java 8 en [el siguiente enlace](#) y ejecutar el archivo .jar (con click derecho por seguridad).
- **Ubuntu:** Seguir [el siguiente tutorial](#).

* Pueden instalar versiones superiores de Java, la versión 8 es la mínima requerida.



Actividad en clase - Ejercicio de Assembly RISC-V

Ejercicio en clases - Parte 1

Complete el programa `median-incomplete.asm` para computar correctamente la mediana de un arreglo ordenado de números.

```
.data
arr: .word 1, 2, 3, 4, 5, 6, 7, 8, 9, 10
len: .word 10
.text
main:
    lw a0, len
    la a1, arr
    addi s0, zero, 2          # Constante para obtener promedios y restos
    jal ra, find_median       # find_median(len, puntero_arr)
    addi a7, zero, 1          # Imprimir resultado
    ecall
    addi a7, zero, 10         # Terminar programa
    ecall
find_median:
    rem t0, a0, s0            # t0 = a0 % s0 = len % 2
    div t1, a0, s0            # t1 = a0 // s0 = len // 2
    beq t0, zero, even_length # Si t0 = 0, entonces el arreglo es par
odd_length:
    add a0, zero, t1          # a0 = indice de la mediana
    jal zero, end
even_length:
    add a0, zero, t1          # a0 = indice del segundo elemento de la mediana
    addi a0, a0, -1           # a0 = indice del primer elemento de la mediana
end:
    jalr zero, 0(ra)          # Se retorna la mediana en el registro a0
```

Actividad en clase - Ejercicio de Assembly RISC-V

Ejercicio en clases - Parte 2

Desarrolle un programa en RISC-V que determine si un *input* N definido en el segmento `.data` es primo o no. El programa debe imprimir un 1 si lo es y un 0 en caso contrario. **Debe seguir la convención de llamada de RISC-V.**

Actividad en clase - Ejercicio de Assembly RISC-V

Ejercicio en clases - Parte 3

Extienda el programa anterior para que el programa imprima 1 si el input N es **primo gemelo**, *i.e.*:

- Es primo.
- Existe otro número primo M tal que $|N - M| = 2$.

Debe seguir la convención de llamada de RISC-V.

Arquitectura RISC-V - Consideraciones respecto a RARS

- En teoría, RISC-V no otorga forma de cargar los valores de memoria de variables directamente en registros, pero RARS sí lo permite.

```
.data
N:      .word 23
.text
# Comportamiento esperado
main:
    la a0, N           # a0 = dirección N
    lw a0, 0(a0)       # a0 = Mem[a0] = N
```

```
.data
N:      .word 23
.text
# Manejo en RARS
main:
    lw a0, N           # a0 = N
```

- Al ejecutar la pseudo-instrucción **call**, RARS almacena los 24 bits más significativos del literal offset en el registro **t1**. Por lo tanto, al definir subrutinas recursivas se sugiere **no operar con este registro**.

```
.text
call example
mv a0, t1
li a7, 1
ecall
li a7, 10
ecall
example:
    ret
```



Arquitectura RISC-V - Consideraciones respecto a RARS

- Podemos ejecutar saltos incondicionales con instrucciones de *branching* o con **jal**.

```
.text
# Salto con branching
main:
    beq zero, zero, end
    addi t0, zero, 1
end:
```

```
.text
# Salto con jal
main:
    jal zero, end
    addi t0, zero, 1
end:
```

- En esta clase todos los ejemplos usan **jal zero, label**, pero ambas formas son correctas.

Errores comunes

A continuación, veremos los errores más comunes que cometen en la actividad de programación de forma que puedan evitarlos.

PEBCAC



**Problem Exists Between
Chair and Computer**

Errores comunes

Uso de instrucciones inexistentes

- Por desconocimiento, se suelen usar instrucciones que no existen o que su nomenclatura es incorrecta, lo que gatilla errores de compilación.
- A continuación, algunos ejemplos reales.

Errores comunes

Uso de instrucciones inexistentes

```
add t0, t9, t10 # t9, t10 no existen
subi t0, t0, 1  # Inst. subi no existe
lw t0, (t1)     # Falta offset
lw t0, t1(4)    # Reg. y offset intercambiados
lw t0, 1(t1)    # Offset no múltiplo de 4
addi zero, t0, 1 # zero no se sobrescribe
```

INCORRECTO

Se usan instrucciones con registros inexistentes o con nomenclatura incorrecta.

```
add t0, t5, t6  # t5, t6 no existen
addi t0, t0, -1 # Resta de lit. con addi y -
lw t0, 0(t1)    # Con offset
lw t0, 4(t1)    # Reg. y offset correctos
lw t0, 8(t1)    # Offset múltiplo de 4
addi t0, zero, 1 # zero se usa como operando
```

CORRECTO

Se usan solo registros definidos con la nomenclatura correcta.

Tip: Revisar el panel derecho de RARS para ver los registros que se pueden utilizar, y presionar el ícono  para ver la lista completa de instrucciones y ejemplos de uso.

Errores comunes

Falta de restauración del *stack pointer*

- Cada vez que se realizan respaldos en la memoria de *stack*, el *stack pointer* decrementa según la cantidad de registros a respaldar.
- Al recuperar los registros, es importante **restaurar el *stack pointer* a su valor original**. En otro caso, el *stack* seguirá incrementando de forma indefinida, lo que puede causar *stack overflow* en llamadas recursivas o inconsistencias en llamados anidados.

Errores comunes

Falta de restauración del *stack pointer*

```
addi sp, sp, -4 # Reserva espacio para 1 reg
sw ra, 0(sp)    # Respalda ra
jal ra, func    # Llamado a func
lw ra, 0(sp)    # Recupera ra
addi a7, zero, 10 # Termina programa
ecall
```

INCORRECTO

No se restaura el *stack pointer* posterior al retorno.

```
addi sp, sp, -4 # Reserva espacio para 1 reg
sw ra, 0(sp)    # Respalda ra
jal ra, func    # Llamado a func
lw ra, 0(sp)    # Recupera ra
addi sp, sp, 4   # Restaura sp
addi a7, zero, 10 # Termina programa
ecall
```

CORRECTO

El *stack pointer* recupera su valor original posterior al llamado.

Errores comunes

Orden erróneo al recuperar registros y ajustar *stack pointer*

- Si por error se ajusta el *stack pointer* y **después** se recuperan los registros, no se estará usando el valor base desde el cual se hicieron los respaldos, lo que implica que no se obtendrán los valores originales.

Errores comunes

Orden erróneo al recuperar registros y ajustar *stack pointer*

```
addi sp, sp, -4 # Reserva espacio para 1 reg
sw ra, 0(sp)    # Respalda ra
jal ra, func    # Llamado a func
addi sp, sp, 4  # Restaura sp
lw ra, 0(sp)    # Recupera ra
addi a7, zero, 10 # Termina programa
ecall
```

INCORRECTO

El *stack pointer* recupera su valor original **antes** de recuperar los registros.

```
addi sp, sp, -4 # Reserva espacio para 1 reg
sw ra, 0(sp)    # Respalda ra
jal ra, func    # Llamado a func
lw ra, 0(sp)    # Recupera ra
addi sp, sp, 4  # Restaura sp
addi a7, zero, 10 # Termina programa
ecall
```

CORRECTO

El *stack pointer* recupera su valor original **después** de recuperar los registros.

Ejercicios

Ahora, veremos algunos ejercicios.

Estos se basan en preguntas de tareas y pruebas de semestres anteriores, por lo que nos servirán de preparación para las evaluaciones.



Ejercicios

Indique justificadamente si el siguiente fragmento de código respeta la convención de llamada de RISC-V.

```
.data
N:      .word 23
divisor: .word 12
.text
main:
    addi sp, sp, -4
    sw ra, 0(sp)
    call remainder
    lw ra, 0(sp)
    addi sp, sp, 4
    j end
remainder:
    lw a0, N
    lw a1, divisor
    rem a0, a0, a1
    ret
end:
```

Examen, 2023-1

Ejercicios

Indique si el siguiente fragmento de código en RISC-V termina su ejecución. Si lo hace, indique el valor del registro `a0`. En otro caso, justifique por qué no termina.

```
.data
N: .word 4
.text
main:
    addi sp, sp, -4
    sw   ra, 0(sp)
    lw   t0, N
    mv   a0, t0
    call factorial
    lw   ra, 0(sp)
    addi sp, sp, 4
    j end
factorial:
    addi sp, sp, -4
    sw   a0, 0(sp)
    blez a0, factorial_zero
    addi a0, a0, -1
    call factorial
    lw   t0, 0(sp)
    mul  a0, a0, t0
    j factorial_end
factorial_zero:
    li   a0, 1
factorial_end:
    addi sp, sp, 4
    ret
end:
```

Examen, 2023-1

Ejercicios

Para esta pregunta, deberá programar una pequeña versión de juego FizzBuzz en RISC-V. Para esto, recibirá en la sección `.data` una etiqueta `N`, que corresponde a un entero positivo y deberá escribir, a partir de la etiqueta `out`, todos los números entre 0 y `N` como `.word`, con la salvedad de que si el número es divisible por 3, este se reemplaza por 70 (ASCII “F”); en caso de ser divisible por 5, se reemplaza por 66 (ASCII “B”); y si es divisible por ambos, se reemplaza por 7066. Además, si un número es primo gemelo, deberá reemplazarlo por 80 (ASCII “P”). A modo de ejemplo, si $N = 15$ la secuencia que se guardará en `out` será: 7066, 1, 2, 80, 4, 80, 70, 80, 8, 70, 66, 80, 70, 80, 14, 7066.

Tarea 3, 2023-1

Antes de terminar

¿Dudas?

¿Consultas?

¿Inquietudes?

¿Comentarios?





DCC

DEPARTAMENTO DE CIENCIA
DE LA COMPUTACIÓN

IIC2343

Arquitectura de Computadores

Clase 8 - Arquitectura RISC-V

Profesor: Germán Leandro Contreras Sagredo

Anexo - Manejo de floats

Además de las instrucciones y registros anteriores, existe la extensión **RV32F** que otorga soporte operaciones sobre números de punto flotante bajo el estándar IEEE754. A continuación, se listan los 32 registros que se añaden con esta extensión.

Registro(s)	Mnemotecnia ABI	Descripción
f0-7, f28-31	ft0-7, ft8-11	Registros temporales de punto flotante. Pierden su valor entre llamados de subrutinas.
f8-9, f18-27	fs0-1, fs2-11	Registros guardados de punto flotante (<i>saved</i>). Preservan su valor entre llamados de subrutinas.
f10-17	fa0-7	Registros de punto flotante para argumentos de subrutinas.
f10-11	fa0-7	Si bien son de argumentos de subrutinas, también se utilizan para almacenar valores de retorno.

* Se consideran *caller-saved* o *callee-saved* bajo los mismos criterios que los registros de números enteros.

Anexo - Manejo de floats

A continuación, se listan las instrucciones **más relevantes** que añade la extensión RV32F.

Mnemotecnia	Instrucción	Tipo	Descripción
FLW rd, imm12(rs1)	Cargar <i>word</i> de tipo float (32 bits)	I	$rd \leftarrow \text{mem}[rs1 + \text{imm12}]$
FSW rs2, imm12(rs1)	Almacenar <i>word</i> de tipo float (32 bits)	S	$rs2 \rightarrow \text{mem}[rs1 + \text{imm12}]$
FADD.S rd, rs1, rs2	Adición de floats	R	$rd \leftarrow rs1 + rs2$
FSUB.S rd, rs1, rs2	Sustracción de floats	R	$rd \leftarrow rs1 - rs2$
FMUL.S rd, rs1, rs2	Multiplicación de floats	R	$rd \leftarrow rs1 * rs2$
FDIV.S rd, rs1, rs2	División de floats	R	$rd \leftarrow rs1 / rs2$
FSQRT.S rd, rs1	Raíz cuadrada de float	R	$rd \leftarrow \sqrt{rs1}$
FCVT.S.W rd, rs1	Convierte int a float	R	$rd \leftarrow \text{float}(rs1)$
FCVT.W.S rd, rs1	Convierte float a int	R	$rd \leftarrow \text{int}(rs1)$

Anexo - Manejo de floats

Ejemplo: Cómputo de la raíz cuadrada de un número N . Si el valor posee decimales, se truncan al transformar el número de vuelta a entero.

```
.data
N:      .word 25
N_trunc: .word 37
.text
main:
    lw t0, N          # t0 = N
    fcvts.w ft0, t0    # ft0 = float(t0)
    fsqrt.s ft0, ft0   # ft0 = sqrt_2(ft0)
    fcvts.w t0, ft0    # t0 = int(ft0)
    add a0, zero, t0   # a0 = t0
    addi a7, zero, 1   # print(a0)
    ecall
    lw t0, N_trunc     # t0 = N_trunc
    fcvts.w ft0, t0    # ft0 = float(t0)
    fsqrt.s ft0, ft0   # ft0 = sqrt_2(ft0)
    fcvts.w t0, ft0    # t0 = int(ft0)
    add a0, zero, t0   # a0 = t0
    ecall              # print(a0)
    addi a7, zero, 10  # exit
    ecall
```


Anexo - Resolución de ejercicios

¡Importante!

Estos ejercicios pueden tener más de un desarrollo correcto. Las respuestas a continuación no son más que soluciones que **no excluyen** otras alternativas igual de correctas.



Actividad en clase - Respuesta

Ejercicio en clases - Parte 1

Desarrolle un programa en RISC-V que determine si un *input* N definido en el segmento `.data` es primo o no. El programa debe imprimir un 1 si lo es y un 0 en caso contrario. **Debe seguir la convención de llamada de RISC-V.**

Respuesta en la siguiente diapositiva.

Actividad en clase - Respuesta

```
.data
N:          .word 17
.text
main:
    lw a0, N           # a0 = N = parameter
    li s0, 1           # Constant s0 = 1
    li s1, 2           # Constant s1 = 2
    addi sp, sp, -4     # Save ra
    sw ra, 0(sp)
    jal ra, check_is_prime
    lw ra, 0(sp)        # Restore ra and stack
    addi sp, sp, 4
    addi a7, zero, 1    # print
    ecall
    addi a7, zero, 10   # exit
    ecall
check_is_prime:
    bge s0, a0, is_not_prime # a0 <= 1 -> is_not_prime
    beq a0, s1, is_prime    # a1 == 2 -> is_prime
    # Check divisors from 2 to sqrt_2(N).
    fcvts.s.w ft0, a0
    fsqrt.s ft0, ft0
    fcvts.w.s t1, ft0
    addi t0, zero, 2        # Starting divisor. t0 = 2
division_loop:
    rem t2, a0, t0          # If a0 % t0 == 0 -> Divisible, not prime.
    beq t2, zero, is_not_prime
    addi t0, t0, 1          # t0 += 1
    bge t1, t0, division_loop # While t0 <= t1, keep looking for divisors.
is_prime:
    addi a0, zero, 1
    jal zero, end
is_not_prime:
    addi a0, zero, 0
end:
    jalr zero, 0(ra)
```

Actividad en clase - Respuesta

Ejercicio en clases - Parte 2

Extienda el programa anterior para que el programa imprima 1 si el input N es **primo gemelo**, *i.e.*:

- Es primo.
- Existe otro número primo M tal que $|N - M| = 2$.

Debe seguir la convención de llamada de RISC-V.

Respuesta en la siguiente diapositiva.

Actividad en clase - Respuesta

```
.data
N:                .word 19
.text
main:
    lw a0, N           # a0 = N = parameter
    addi s0, zero, 1   # Constant s0 = 1
    addi s1, zero, 2   # Constant s1 = 1
    addi sp, sp, -4    # Save ra
    sw ra, 0(sp)
    jal ra, check_is_twin_prime
    lw ra, 0(sp)        # Restore ra and stack
    addi sp, sp, 4
    addi a7, zero, 1   # print
    ecall
    addi a7, zero, 10  # exit
    ecall

check_is_twin_prime:
    addi sp, sp, -8    # Save ra and a0
    sw ra, 0(sp)
    sw a0, 4(sp)
    jal ra, check_is_prime
    bne a0, s0, is_not_twin_prime
    lw a0, 4(sp)
    addi a0, a0, -2
    jal ra, check_is_prime
    beq a0, s0, is_twin_prime
    lw a0, 4(sp)
    addi a0, a0, 2
    call check_is_prime
    beq a0, s0, is_twin_prime
    # Else: N is not twin prime
    is_not_twin_prime:
        addi a0, zero, 0
        jal zero, end
    is_twin_prime:
        addi a0, zero, 1
    end:
        lw ra, 0(sp)    # Restore ra and stack
        addi sp, sp, 8
        jalr zero, 0(ra)
```

```
check_is_prime:
    bge s0, a0, is_not_prime    # a0 <= 1 -> is_not_prime
    beq a0, s1, is_prime        # a0 == 2 -> is_prime
    # Check divisors from 2 to sqrt_2(N).
    fcvt.s.w ft0, a0
    fsqrt.s ft0, ft0
    fcvt.w.s t1, ft0
    addi t0, zero, 2            # Starting divisor. t0 = 2
    division_loop:
        rem t2, a0, t0          # If a0 % t0 == 0 -> Divisible, not prime.
        beq t2, zero, is_not_prime
        addi t0, t0, 1          # t0 += 1
        bge t1, t0, division_loop    # While t0 <= t1, keep looking for divisors.
    is_prime:
        addi a0, zero, 1
        jal zero, end_check_is_prime
    is_not_prime:
        addi a0, zero, 0
    end_check_is_prime:
        jalr zero, 0(ra)
```

Ejercicios - Respuesta

Indique justificadamente si el siguiente fragmento de código respeta la convención de llamada de RISC-V.

```
.data
N:      .word 23
divisor: .word 12
.text
main:
    addi sp, sp, -4
    sw ra, 0(sp)
    call remainder
    lw ra, 0(sp)
    addi sp, sp, 4
    j end
remainder:
    lw a0, N
    lw a1, divisor
    rem a0, a0, a1
    ret
end:
```

Respuesta en la siguiente diapositiva.

Ejercicios - Respuesta

El fragmento de código **no respeta** la convención de RISC-V ya que los registros `a0`, `a1` se usan como argumentos de la subrutinas, por lo que deberían cargar sus valores **antes** del llamado de la subrutina y no dentro de ella.

Ejercicios - Respuesta

Indique si el siguiente fragmento de código en RISC-V termina su ejecución. Si lo hace, indique el valor del registro `a0`. En otro caso, justifique por qué no termina.

```
.data
N: .word 4
.text
main:
    addi sp, sp, -4
    sw   ra, 0(sp)
    lw   t0, N
    mv   a0, t0
    call factorial
    lw   ra, 0(sp)
    addi sp, sp, 4
    j    end
factorial:
    addi sp, sp, -4
    sw   a0, 0(sp)
    blez a0, factorial_zero
    addi a0, a0, -1
    call factorial
    lw   t0, 0(sp)
    mul  a0, a0, t0
    j    factorial_end
factorial_zero:
    li   a0, 1
factorial_end:
    addi sp, sp, 4
    ret
end:
```

Respuesta en la siguiente diapositiva.

Ejercicios - Respuesta

El programa anterior **no termina** su ejecución ya que posee llamadas recursivas que **no respaldan** la dirección de retorno del registro ra. Por dicho motivo, el programa podría no terminar o arrojar errores.

Ejercicios - Respuesta

Para esta pregunta, deberá programar una pequeña versión de juego FizzBuzz en RISC-V. Para esto, recibirá en la sección `.data` una etiqueta `N`, que corresponde a un entero positivo y deberá escribir, a partir de la etiqueta `out`, todos los números entre 0 y `N` como `.word`, con la salvedad de que si el número es divisible por 3, este se reemplaza por 70 (ASCII “F”); en caso de ser divisible por 5, se reemplaza por 66 (ASCII “B”); y si es divisible por ambos, se reemplaza por 7066. Además, si un número es primo gemelo, deberá reemplazarlo por 80 (ASCII “P”). A modo de ejemplo, si `N = 15` la secuencia que se guardará en `out` será: 7066, 1, 2, 80, 4, 80, 70, 80, 8, 70, 66, 80, 70, 80, 14, 7066.

Respuesta en la siguiente diapositiva.

Ejercicios - Respuesta

```
.globl start
.data
N: .word 15
.text
start:
    lw s1, N           # max number
    li s0, 0           # counter
    li s2, 3           # first divisor
    li s3, 5           # second divisor
    while:
        mv a0, s0
        call is_twin_prime # is_twin_prime(a0 = counter). No need to save t* registers
        bnez a0, twin_prime
        rem t0, s0, s2     # t0 = s0 % 3 = remainder of division by 3
        rem t1, s0, s3     # t1 = s0 % 5 = remainder of division by 5
        beq t1, t0, fizzbuzz # t0 == t1 -> s0 divisible by 3 and 5? -> FizzBuzz
        beq t0, zero, fizz # t0 == 0 -> s0 divisible by 3 -> Fizz
        beq t1, zero, buzz # t1 == 0 -> s0 divisible by 5 -> Buzz
        fizzbuzz:
            bnez t1, skip # t1 != 0 -> s0 not divisible by 3 nor 5
            li a0, 7066
            j continue
        fizz:
            li a0, 70
            j continue
        buzz:
            li a0, 66
            j continue
        twin_prime:
            li a0, 80
            j continue
        skip:
            mv a0, s0
        continue:
            mv a1, s0
            call store # store(a0 = value_to_store, a1 = counter)
            addi s0, s0, 1 # If s0 > s1 -> counter > N, array is finished
            bgt s0, s1, end
            j while
    store:
        addi sp, sp, -8 # s* registers are callee-saved
        sw s0, 0(sp)
        sw s1, 4(sp)
        la s0, N
        addi s0, s0, 4 # s0 = N variable address
        li s1, 4 # s0 += 4 for first available array address
        mul s1, s1, a1 # s1 = 4 (bytes stored for each array element)
```

```
add s0, s1, s0 # Stored value address = Address(N)+4*counter
sw a0, 0(sp) # Value stored
lw s0, 0(sp) # s* registers restored
lw s1, 4(sp)
addi sp, sp, 8
ret

end:
    li a7, 10
    ecall

is_twin_prime:
    addi sp, sp, -4 # a0 = number to check. a0=1 if twin, a0=0 elsewhen
    sw ra, 0(sp) # We store ra for the final return.
    mv a1, a0
    call is_prime # is_prime(a0=counter). a0=1 if prime, a0=0 elsewhen
    beqz a0, end_is_twin_prime # a0 == 0 if counter is not prime
    addi a0, a1, -2
    call is_prime # is_prime(a0 - 2)
    bgtz a0, end_is_twin_prime # a0 > 0 if counter is twin prime
    addi a0, a1, 2
    call is_prime # is_prime(a0 + 2)
    end_is_twin_prime: # a0 == 1 if N is twin prime else a0 == 0
        lw ra, 0(sp) # We restore ra for correct return
        addi sp, sp, 4
        ret

is_prime:
    li t0, 1
    ble a0, t0, not_prime # Edge case for counter <= 1
    fcvt.s.w ft0, a0 # ft0 = float(a0)
    fsqrt.s ft0, ft0 # ft0 = sqrt_2(ft0)
    fcvt.w.s t0, ft0 # t0 = int(ft0)
    li t1, 2
    while_is_prime_check:
        rem t2, a0, t1 # if a0 % t1 == 0 -> counter not prime
        beqz t2, not_prime
        addi t1, t1, 1
        ble t1, t0, while_is_prime_check

prime:
    li a0, 1
    ret

not_prime:
    li a0, 0
    ret
```